

Non-SARGable Predicates

#sql-server #performance #indexing #optimizer

What SARGable Means

SARG stands for Search ARGument. A predicate is SARGable if the query optimizer can use an index seek to evaluate it. The column must appear naked on one side of the comparison, with no functions, no computations, and no wrapping.

```
-- SARGable: index seek on OrderDate
Where OrderDate >= '2026-01-01'

-- Non-SARGable: function wrapping prevents seek
Where Year(OrderDate) = 2026
```

Both return the same rows. But the first uses an index seek (touching only matching rows), while the second requires a full index scan (touching every row, evaluating the function on each).

Common Non-SARGable Patterns

1. IsNull() in Comparisons

```
-- Non-SARGable
Where IsNull(Col1, 0) = IsNull(Col2, 0)

-- The optimizer can't seek on either column because
-- both are wrapped in function calls
```

2. Functions on Columns

```
-- Non-SARGable
Where Convert(Date, CreatedDtTm) = '2026-03-21'
Where Upper(LastName) = 'SMITH'
Where DateDiff(Day, ShipDate, GetDate()) > 30

-- SARGable equivalents
Where CreatedDtTm >= '2026-03-21' And CreatedDtTm < '2026-03-22'
Where LastName = 'SMITH' -- (if collation is case-insensitive, which is default)
Where ShipDate < DateAdd(Day, -30, GetDate())
```

3. Arithmetic on Columns

```
-- Non-SARGable
Where Price * Quantity > 1000

-- SARGable (if you can isolate the column)
Where Price > 1000.0 / Quantity -- still not great, but better
```

4. OR with NULL Checks

```
-- Non-SARGable
Where (IsMovedToFile Is Null Or IsMovedToFile <> 1)

-- This is really "anything that isn't 1".
-- Best fix: a filtered index.
Create Index IX_NotMoved On ImgImage(Id)
    Where IsMovedToFile Is Null Or IsMovedToFile <> 1
```

5. Implicit Conversions

```
-- Column is varchar(10), parameter is nvarchar
Where ProductCode = @ProductCode -- @ProductCode is nvarchar

-- SQL Server must convert EVERY row's varchar to nvarchar
-- because nvarchar has higher precedence.
-- Fix: declare @ProductCode as varchar, or explicitly cast the parameter.
```

How to Detect Non-SARGable Predicates

In the Execution Plan

- Look for Index Scan where you expect Index Seek.
- Check the Predicate property on the Scan operator. If it contains function calls, that is the culprit.
- Yellow warning triangles for implicit conversions.

In SET STATISTICS IO

- A table shows reads proportional to its total size rather than the expected selectivity.
- Example: a table with 10M rows shows 40,000 reads for a query that should match 100 rows.

Systematic Code Review

Search for these patterns in your WHERE clauses:

```
IsNull(column, ...)
Convert(... column ...)
Cast(column ...)
Upper(column)
Lower(column)
DateDiff(... column ...)
Year(column)
Month(column)
```

The IsNull Dilemma

A common pattern in pharmacy systems:

```
Where IsNull(EP.Col1, 0) = IsNull(EP.Col2, 0)
```

This handles NULLs on both sides, treating NULL = NULL as true (which standard SQL does not). Replacing this with SARGable code changes the semantics:

```
-- NOT semantically equivalent:  
Where EP.Col1 = EP.Col2
```

```
-- This treats NULL = NULL as UNKNOWN (false), which is different
```

When the IsNull pattern is comparing two columns in the same table (not a parameter), and both can be NULL, there is no clean SARGable equivalent. The fix is typically a computed column with an index:

```
Alter Table EP Add ComparedVal As IsNull(Col1, 0) - IsNull(Col2, 0) Persisted  
Create Index IX_ComparedVal On EP(ComparedVal) Where ComparedVal = 0
```

But this is an index and schema change, not a query change. In our refactoring work, we note these patterns but don't change them when the semantic difference matters.

Real-World Example

In `Isp_RxfGetListOfManualFillGroups` (v48 to v49), we identified `IsNull(0.PriCodeSys, 0) = IsNull(P.PriCodeSys, 0)` as non-SARGable. We documented it in the changelog under "Noted, NOT Changed" because replacing it with a plain `=` comparison would change the NULL-handling semantics, potentially breaking the business logic for orders or pharmacies with NULL priority codes.

The Pragmatic Approach

Not every non-SARGable predicate needs to be fixed:

1. If the table is small (under 10,000 rows), a scan is fine. Don't over-optimize.
2. If the predicate is on a joined table (not the driving table), the scan cost may be negligible.
3. If fixing it changes semantics, document it and move on. Correctness trumps performance.
4. If a filtered index can help, recommend it as a separate schema change with its own testing cycle.

Related Concepts

- **Parameter Sniffing**: even with SARGable predicates, the wrong sniffed value can produce scans.
- **Density Vector**: the statistics the optimizer uses for SARGable seek estimates.
- **LEFT JOIN OR Anti-Pattern**: OR conditions are inherently harder to make SARGable.
- **FORCESEEK Hints**: forces seeks even on non-SARGable predicates (but the engine adds a residual filter, which may not be faster).